The Bootstrap

Mark M. Fredrickson (mfredric@umich.edu)

Module 3: Resample

Computational Methods in Statistics and Data Science (DATASCI 406)

Overview of Module 3

Modules 1 and 2

In module 1 we used samplers R provides to

- estimate integrals, expectations and probabilities,
- compute **Monte Carlo** *p*-**values** and invert tests into intervals,
- check an interval's **coverage** by simulation.

In module 2 we wrote samplers for distributions R does not have.

Both modules sampled from a distribution **we chose**, such as a null hypothesis or a model.

Module 3 overview

In this module the only distribution we have is **the one the data give us**.

- **Bootstrap** (today): resample the observations with replacement to get a standard error or an interval.
- **Jackknife and cross-validation**: leave observations out to estimate bias, standard errors and prediction error.
- **Structured data**: resample blocks of a time series, or residuals in a regression.
- **Permutation tests**: rearrange signs or group labels under a null hypothesis.

The Problem

Lead in Flint's water

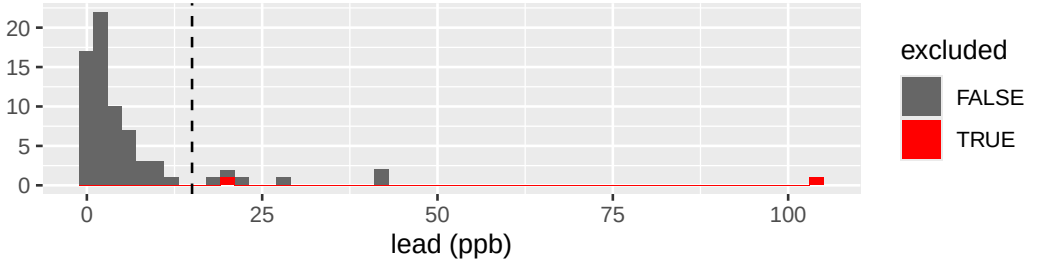
Under the federal Lead and Copper Rule, a water system must act if more than 10% of sampled homes have lead above 15 parts per billion (ppb), that is, if the **90th percentile** of the samples is above 15 ppb.

In 2015 the Michigan Department of Environmental Quality (MDEQ) collected 71 samples from homes in Flint.

```
mdeq <- read.csv("../../data/flint_mdeq.csv")
x <- mdeq$lead
n <- length(x)
quantile(x, 0.9)
```

```
## 90%
##  18
```

Two excluded samples



MDEQ excluded two samples from its calculation, one from a house with a filter and one from a business. Without them the 90th percentile is 11.4 ppb, below the action level.

Is a change from 18 to 11.4 large compared to the uncertainty in a 90th percentile from 71 homes?

Sampling distributions

Model the samples as $X_1, \ldots, X_n \overset{\text{iid}}{\sim} F$. We estimate a parameter θ, here the 90th percentile of F, with a statistic $T = T(X_1, \ldots, X_n)$.

To get a standard error or a confidence interval we need the **sampling distribution of** T: the distribution of T over repeated samples of size n from F. There are three routes to it.

1. **Derive it** from assumptions about F, as for $\bar{X}$ with normal data.
2. Use a **large sample approximation**, such as the CLT.
3. **Simulate it**: draw many samples from F and compute T on each.

Inversion Method for Estimated Distributions

Estimating the CDF

The **cumulative distribution function** F is an expectation:

$$F(t) = P(X \leq t) = \mathsf{E}\left(\mathbf{1}(X \leq t)\right).$$

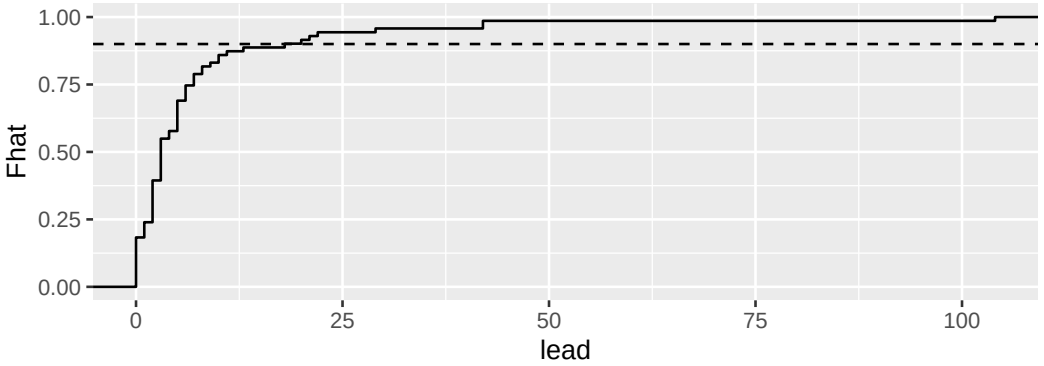
We estimate an expectation with a sample mean, which gives the **empirical CDF**:

$$\hat{F}(t) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(X_i \leq t).$$

$\hat{F}(t)$ is the fraction of observations at or below t. It is the CDF of a discrete distribution that puts mass $1/n$ on each observation.

The empirical CDF of the MDEQ samples

```
ggplot(mdeq, aes(lead)) + stat_ecdf(geom = "step") +
  geom_hline(yintercept = 0.9, lty = 2) + labs(y = "Fhat")
```



Inverting $\hat{F}$

The inverse transform (module 2, day 1) draws $U \sim U(0,1)$ and applies a quantuile fuction to U.

We will need the quantile function of $\hat{F}$, $\hat{Q}$.

Sort the observations, $x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}$. $\hat{F}$ jumps by $1/n$ at each one, so

$$\hat{Q}(u) = x_{(j)} \quad \text{when} \quad \frac{j-1}{n} < u \leq \frac{j}{n}.$$

Claim: $\hat{Q}(U)$ picks one of the n observations uniformly at random,

$$P(\hat{Q}(U) = x_{(j)}) = \frac{1}{n}, \quad j = 1, \ldots, n.$$

Proof

Recall $P(a < U \leq b) = b - a$ for $0 \leq a \leq b \leq 1$.

Condition on the observed sample and find

$$P(\hat{Q}(U) = x_{(j)}) = P\left(\frac{j-1}{n} < U \leq \frac{j}{n}\right) = \frac{j}{n} - \frac{j-1}{n} = \frac{1}{n}.$$

If a value appears k times, such as the 13 samples with no detectable lead, $\hat{F}$ jumps by k/n there. The same argument gives that value probability k/n, which is what drawing a uniform index into $x_1, \ldots, x_n$ gives too.

Drawing n independent values from $\hat{F}$ is therefore **sampling n observations with replacement**.

In R

The quantile function of $\hat{F}$ evaluated at a uniform draw, $\hat{Q}(u) = x_{(\lceil nu \rceil)}$:

```
qhat <- function(u) sort(x)[ceiling(n * u)]
qhat(runif(10))
```

```
##  [1]  1 20  0  3  0  3 13  8  8  6
```

The same distribution, with `sample()`:

```
sample(x, size = 10, replace = TRUE)
```

```
##  [1]  2  6 22  2  2  0  0  0  7  5
```

The Bootstrap

The algorithm

For $b = 1, \ldots, B$:

1. draw a **bootstrap sample** $X_1^*, \ldots, X_n^*$ from $x_1, \ldots, x_n$ with replacement,
2. compute $T_b^* = T(X_1^*, \ldots, X_n^*)$.

The **bootstrap distribution** of $T_1^*, \ldots, T_B^*$ estimates the sampling distribution of T. In particular, the **bootstrap standard error** is

$$\widehat{\mathrm{SE}}(T) = \sqrt{\frac{1}{B-1} \sum_{b=1}^{B} \left(T_b^* - \bar{T}^* \right)^2}.$$

How many bootstrap samples are there?

Each of the n draws can be any of the n observations, so there are n^n ordered bootstrap samples.

```
n^n
```

```
## [1] 2.75e+131
```

We cannot compute T on all of them, so we take a Monte Carlo sample of B of them. This is module 1's estimation, applied to the distribution of T^*.

Two sources of error

The bootstrap SE differs from the true SE of T for two reasons.

1. **Using $\hat{F}$ for F.** This is fixed once the data are collected, and it shrinks only as n grows.
2. **Monte Carlo error.** We use B samples instead of all n^n. This shrinks as B grows, and we control it.

Increasing B removes the second error and does nothing about the first.

Choosing B

Compute the bootstrap SE of $\bar{X}$ 20 times at each B:

```
boot_se <- function(B) {
  sd(replicate(B, mean(sample(x, replace = TRUE))))
}
se_100 <- replicate(20, boot_se(100))
se_2000 <- replicate(20, boot_se(2000))
c(B100 = sd(se_100), B2000 = sd(se_2000))
```

```
##     B100    B2000
## 0.12983 0.02473
```

At $B = 2000$ the SE varies by about 1.5% between runs. Intervals use the tails of the bootstrap distribution, which need larger B than the SE.

Example: the 90th Percentile

Checking against the mean

For $\bar{X}$ we know the answer, so we check the method there first.

```
B <- 4000
boot_mean <- replicate(B, mean(sample(x, replace = TRUE)))
c(bootstrap = sd(boot_mean), formula = sd(x) / sqrt(n))
```

```
## bootstrap   formula
##     1.698     1.703
```

As $B \to \infty$ the bootstrap SE of $\bar{X}$ goes to $\hat{\sigma}/\sqrt{n}$, where $\hat{\sigma}^2$ divides by n rather than $n-1$. Here that factor is $\sqrt{(n-1)/n} = 0.993$.

The 90th percentile has no simple SE formula

For large n, the sample p quantile has approximate variance

$$\frac{p(1-p)}{n\, f(\xi_p)^2},$$

where ξ_p is the true p quantile and f is the density of F.

Using this formula requires estimating the density of lead levels at the 90th percentile, from the few samples near it. The bootstrap needs no density.

Bootstrapping the 90th percentile

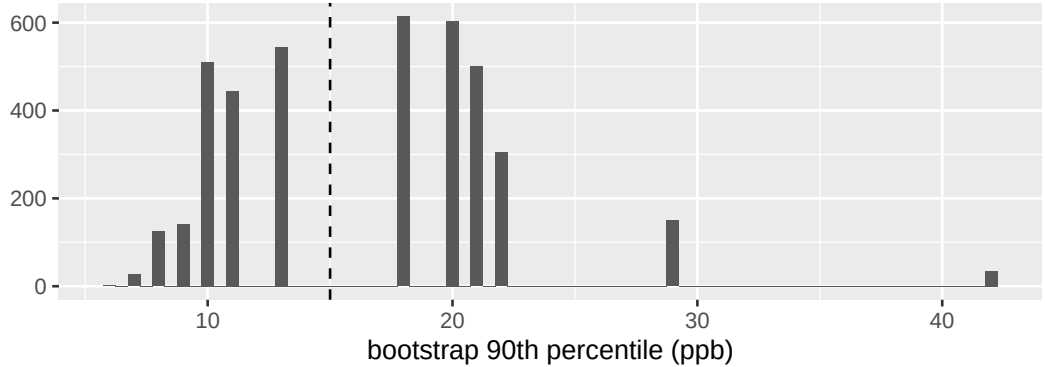
The code is the same, with a different statistic:

```
p90 <- function(z) quantile(z, 0.9, names = FALSE)
t_obs <- p90(x)
boot_p90 <- replicate(B, p90(sample(x, replace = TRUE)))
sd(boot_p90)
```

```
## [1] 5.821
```

The SE is 5.8 ppb, about the size of the 6.6 ppb change from excluding the two samples.

A lumpy bootstrap distribution



The 90th percentile of a bootstrap sample is one of the largest observations, or an average of two neighboring ones. The 4000 bootstrap values take only 13 distinct values.

With and without the excluded samples

```
boot_excl <- replicate(B, p90(sample(x_excl, replace = TRUE)))
rbind(all = c(T = t_obs, SE = sd(boot_p90)),
      excluded = c(T = p90(x_excl), SE = sd(boot_excl)))
```

```
##              T    SE
## all       18.0 5.821
## excluded 11.4 4.806
```

Removing two of the largest values lowers both the estimate and its SE.

From $T_1^*, \ldots, T_B^*$ we can estimate the **standard error** of T, **quantiles** of its sampling distribution, which give confidence intervals, and the **bias** of T (day 2).

Bootstrap Confidence Intervals

The normal interval

If T is approximately $N(\theta, \mathrm{SE}^2)$, a $1-\alpha$ interval is $T \pm z_{1-\alpha/2}\,\mathrm{SE}$. Use the bootstrap SE:

$$T \pm z_{1-\alpha/2}\,\widehat{\mathrm{SE}}(T).$$

```
(normal_ci <- t_obs + c(-1, 1) * qnorm(0.975) * sd(boot_p90))
```

```
## [1]  6.59 29.41
```

The interval is centered at T from the original sample, not at $\bar{T}^*$.

The basic interval

We want a and b with $P(a \leq T - \theta \leq b) = 1 - \alpha$, which gives

$$P(T - b \leq \theta \leq T - a) = 1 - \alpha.$$

The distribution of $T - \theta$ is unknown. The bootstrap approximates it with the distribution of $T^* - T$: the original sample plays the role of the population, and T plays the role of θ.

So $a \approx T^*_{\alpha/2} - T$ and $b \approx T^*_{1-\alpha/2} - T$, where T^*_p is the p quantile of the bootstrap distribution. The **basic interval** is

$$\left[2T - T^*_{1-\alpha/2},\ 2T - T^*_{\alpha/2}\right].$$

The basic interval in R

```
q <- quantile(boot_p90, c(0.025, 0.975))
(basic_ci <- 2 * t_obs - rev(q))
```

```
## 97.5%  2.5%
##     7    28
```

The upper quantile of T^* sets the lower end of the interval. If the bootstrap distribution has a long right tail, the basic interval extends further to the left of T.

The percentile interval

The **percentile interval** uses the quantiles of T^* directly:

$$\left[T^*_{\alpha/2},\ T^*_{1-\alpha/2}\right].$$

```
(percentile_ci <- q)
```

```
##  2.5% 97.5%
##     8    29
```

The percentile interval is the basic interval reflected about T, and the two agree when the bootstrap distribution is symmetric about T. The percentile interval is also correct when some increasing function g makes $g(T) - g(\theta)$ symmetric, and we never need to know g.

All three intervals

```
three_cis <- function(t, boot) {
  q <- quantile(boot, c(0.025, 0.975), names = FALSE)
  rbind(normal = t + c(-1, 1) * qnorm(0.975) * sd(boot),
        basic = 2 * t - rev(q), percentile = q)
}
three_cis(t_obs, boot_p90)
```

```
##            [,1]  [,2]
## normal     6.59 29.41
## basic      7.00 28.00
## percentile 8.00 29.00
```

With all 71 samples the three intervals are similar.

Without the excluded samples

```
three_cis(p90(x_excl), boot_excl)
```

```
##              [,1]  [,2]
## normal       1.98 20.82
## basic       -0.60 15.60
## percentile   7.20 23.40
```

Without the two samples the bootstrap distribution is skewed to the right, and the intervals disagree. The basic interval reaches below zero, which lead cannot be.

What the intervals say about Flint

All six intervals contain 15 ppb. With or without the two samples, 71 homes cannot settle whether the 90th percentile is above the action level.

Virginia Tech's citizen science project collected 271 samples:

```
vt <- read.csv("../../data/flint_vt.csv")
boot_vt <- replicate(B, p90(sample(vt$first_draw, replace = TRUE)))
c(T = p90(vt$first_draw), quantile(boot_vt, c(0.025, 0.975)))
```

```
##      T  2.5% 97.5%
## 26.64 16.99 31.06
```

The bootstrap treats each set of homes as a random sample from Flint. If the homes were not chosen at random, resampling them cannot show it.

Better intervals

The normal, basic and percentile intervals have coverage close to $1 - \alpha$ only when n is large. Bootstrap intervals with better coverage in small samples exist:

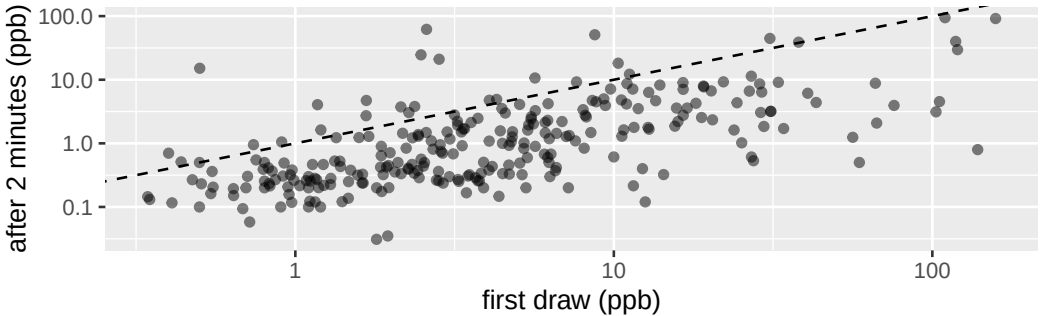
- the **bootstrap-t** interval, which bootstraps a studentized statistic,
- the **BCa** interval, which adjusts the percentile interval for bias and skewness.

The bootstrap-t needs a standard error for every bootstrap sample. The jackknife (day 2) gives one cheaply, and we return to the bootstrap-t then. BCa is in Rizzo §8.4 and is not covered in this course.

Paired Data

Flushing the tap

Each Virginia Tech kit had three bottles. Residents filled the first with the first water out of the tap, and the third after running the tap for 2 minutes.



An observation is now a home, $(X_i, Y_i) =$ (first draw, after flushing).

Resampling rows

$\hat{F}$ puts mass $1/n$ on each **pair**, so a draw from $\hat{F}$ is a row chosen at random.

```
lx <- log(vt$first_draw)
ly <- log(vt$flush_2min)
n_vt <- nrow(vt)
boot_cor <- replicate(B, {
  i <- sample(n_vt, replace = TRUE)
  cor(lx[i], ly[i])
})
c(r = cor(lx, ly), SE = sd(boot_cor))
```

```
##       r      SE
## 0.64760 0.04362
```

How much does flushing help?

The mean of $\log X_i - \log Y_i$ measures the effect of flushing. Its exponential is the ratio of the geometric means.

```
d <- lx - ly
boot_d <- replicate(B, mean(d[sample(n_vt, replace = TRUE)]))
exp(c(est = mean(d), quantile(boot_d, c(0.025, 0.975))))
```

```
##   est  2.5% 97.5%
## 4.170 3.625 4.769
```

Flushing for 2 minutes cuts lead by a factor of about 4.2. Because $\bar{D}$ is a mean, we can check the bootstrap SE, 0.071, against $s_D/\sqrt{n} = 0.072$.

Resampling columns separately

Suppose we resample each bottle on its own:

```
boot_cols <- replicate(B, {
  lx_s <- sample(lx, replace = TRUE)
  ly_s <- sample(ly, replace = TRUE)
  c(cor = cor(lx_s, ly_s), d = mean(lx_s - ly_s))
})
apply(boot_cols, 1, sd)
```

```
##     cor       d
## 0.06064 0.11951
```

This draws X^* and Y^* independently. The correlations center on 0, and the SE of $\bar{D}$ is 0.12 instead of 0.071. Resampling homes keeps each home's two bottles together.

When the Bootstrap Fails

The sample maximum

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim} U(0, \theta)$. The MLE of θ is $T = X_{(n)} = \max_i X_i$.

```
theta <- 1
n_u <- 50
u <- runif(n_u, 0, theta)
boot_max <- replicate(B, max(sample(u, replace = TRUE)))
mean(boot_max == max(u))
```

```
## [1] 0.6295
```

The bootstrap maximum equals the sample maximum in about 63% of bootstrap samples.

Why 63%

$X^*_{(n)} < x_{(n)}$ only when none of the n draws is $x_{(n)}$. Each draw misses it with probability $1 - 1/n$, so

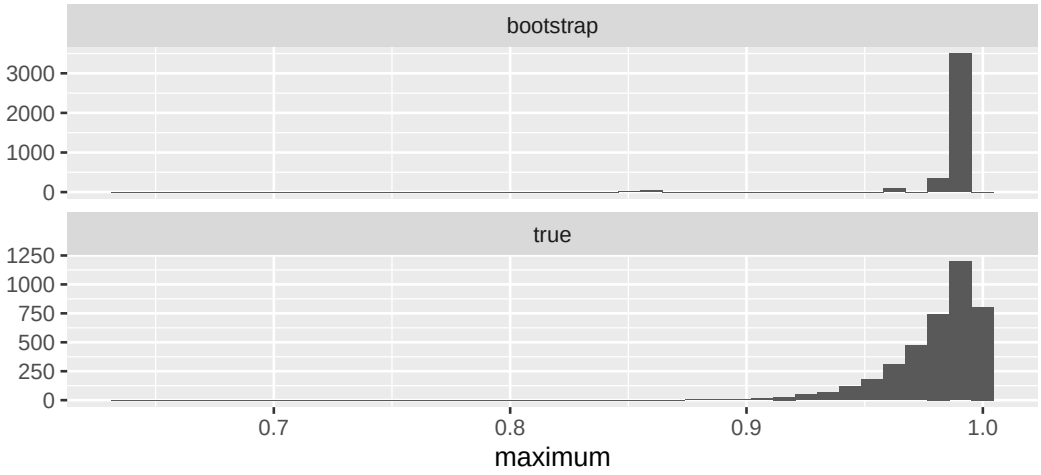
$$P(X^*_{(n)} = x_{(n)}) = 1 - \left(1 - \frac{1}{n}\right)^n \to 1 - e^{-1} \approx 0.632.$$

```
1 - (1 - 1 / n_u)^n_u
```

```
## [1] 0.6358
```

The true sampling distribution of $X_{(n)}$ is continuous, with $P(X_{(n)} \le t) = (t/\theta)^n$. It puts probability zero on any single value.

Bootstrap and true distributions



Coverage

Since $T^* \leq x_{(n)} < \theta$, the percentile interval **never** contains θ.

```
covers <- replicate(1000, {
  u <- runif(n_u, 0, theta)
  bm <- replicate(500, max(sample(u, replace = TRUE)))
  ci <- three_cis(max(u), bm)
  ci[, 1] <= theta & theta <= ci[, 2]
})
rowMeans(covers)
```

```
##     normal      basic percentile
##      0.788      0.883      0.000
```

The maximum depends only on the upper edge of F, and $\hat{F}$ has no mass above $x_{(n)}$.

Rules of thumb: smoothness

The bootstrap works when a small change in F makes a small change in T, so that T computed from $\hat{F}$ behaves like T computed from F.

- **Works well:** smooth functions of means, such as the mean, variance, correlation, and a ratio of means.
- **Works, roughly:** quantiles away from the edges, such as the median or the 90th percentile. Expect a lumpy bootstrap distribution, as in Flint.
- **Fails:** the maximum and minimum; the mean when $\mathrm{Var}(X)$ is infinite; a statistic with a kink at the true value, such as $|\bar{X}|$ when $\mu = 0$.

A spike at T in the bootstrap distribution is a warning sign. When in doubt, check coverage by simulation from a known F.

Summary

Summary

- The bootstrap estimates the sampling distribution of T by drawing samples of size n from $\hat{F}$, which means sampling the observations with replacement.
- Its spread estimates the SE, and its quantiles give the normal, basic and percentile intervals.
- Check the method on a statistic whose SE is known, such as $\bar{X}$.
- With paired data, resample rows. Resampling columns separately breaks the pairing.
- Increasing B reduces Monte Carlo error but not the error from using $\hat{F}$ for F.
- The bootstrap cannot detect how the data were collected, and it fails for the sample maximum.